

Studententuin

Probleem defenitie

Er is te veel keuze uit hosting platforms en veel zijn erg duur dus ik wil een goedkope optie worden met een open mangement ik ga de hele project documenteren en laten zien hoe ik alles maak en er een YT video van maken.

Planning

Voor het globale project inc de video is het plan om het in 6 maanden af te krijgen. De eersete 2 weken ga ik besteden aan het maken van de api voor de userdata en applicaties daarna 2 weken frontend met een doel van 1-2 uur per dag coderen.

Voor de hosting ga ik voor de eerste maand voor een enkele server waarop de backend, frontend en db op zullen runnen deze zal ongeveer 6 dollar per maand kosten waardoor tijdens dev we geen hoge kosten hebben. Voor de uiteindelijke hosting maken we een aparte server waarop de hosting van de aplicaties plaats vind hier ga ik later in het project een keuze in maken.

Eventuele valkuilen zijn dat ik motivatie verlies voor dit project of het druk heb. Het doel wordt om iedere dag minimaal 5 min te coderen en als ik geen zin heb is 5 min zo gedaan maar als je eenmaal begonnen bent is het makkelijker om door te gaan.

Requirements

Epics:

Epic 1: User authentication:

Users moeten zich kunnen authenticaten zodat ze bij hun applicaties kunnen en de logs en env variabelen kunnen zien en zodat ze deze kunnen managen.

De scope van deze epic includeert JWT tokens cookies authguard en verboden pagina's maar ook inloggen, registreren, wachtwoord reseten, account updaten en account verwijderen. Buiten deze scope valt alles wat met applicaties te maken heeft.

Dit is een erg belangrijk deel van dit project ook voor de veiligheid van de user data en dus het is cruciaal dat dit goed en veilig word geïmplementeerd

Deze Epic is klaar als een user zich kan registreren zijn email geverifieerd kan worden, kan inloggen, kan uitloggen, zijn account kan updaten, zijn account kan verwijderen en zijn wachtwoord kan veranderen.

Epic 2: Applicatie CRUD

Users moeten hun eigen applicaties kunnen updaten, deleten, createn, en lezen.

De scope hierin valt simpel weg in het maken en updaten van de applicaties richting de database maar niet wat er met de applicaties gebeurt op de server zelf het zal wel enige aanhang hebben maar alles wat bij het hosten gebeurt valt buiten deze epic

Dit is een van de meest fundamentele dingen van deze applicatie zonder dit is er weinig te doen dit moet ook op een veilige manier maar is minder belangrijk dan de user auth

Deze epic is klaar als een user de applicatie kan updaten, deleten, createn en lezen en ze kan createn op basis van de level wat de user is.

Epic 3: UI

Het maken van een goed uitziend UI en de data hierop tonen is erg belangrijk voor het maken van de applicatie dit is wat de user ziet en waar hij mee interact.

De scope gaat om alle HTML en ook wat er net onder valt dus het injecten van de data naar de HTML dit gedeelte heeft niks te maken met het maken van de API connectie die valt onder de User authenticatie en applicatie CRUD

Dit is ook een erg belangrijk gedeelte van de applicatie en UI moet netjes zijn.

De UI is klaar wanneer alles goed is geïmplementeerd en de UI responsive is en het de WCAG guidelins volgt

Epic 4: Applicatie management

Het daadwerkelijk hosten van de applicaties en het managen van bijvoorbeeld de env variabelen en het laten zien van de stats en logs van de applicaties is erg belangrijk.

Dit heeft alles te maken met wat er op de aparte hosting server gebeurt en via de backend bij de frontend komt maar heeft niks te maken met het opslaan van applicaties in de db.

Dit is ook een van de meest belangrijke punten van deze applicatie en dit gedeelte moet heel erg veilig dus er moet extra veel tijd worden besteed aan het veilig maken van deze applicatie

Dit gedeelte is af als een applicatie gehost kan worden offline gehaald kan worden en de user alles kan zien en aanpassen aan zijn of haar applicatie

Epic 5: Security

Het secure maken van de applicatie is heel erg belangrijk dus we gaan een extra epic maken in het maken van een secure CI/CD het maken van testen het pentesten en updaten van de applicatie hierop en nog veel meer.

In de scope van dit gedeelte valt onder CI/CD het testen maar ook het hosten en veilig maken van het host gedeelte het storen van containers op een veilige plek.

Dit is het belangrijkste van allemaal het veilig maken van de applicatie is cruciaal voor success dus hier mag niks fout gaan.

Het gedeelte is af als ZAP 0 vulnerabilities kan vinden en er geen supply chain attacks zijn en de SAST analyse clean is.

(Deze zullen gemaakt worden in de toekomst voor nu nog niet relevant.)

Epic 6: Email automation

Epic 7: Payment processing

Epic 8: Resource saving

Epic 9: Free websites

User stories:

Epic 1:

1. Users moeten kunnen inloggen
2. Users moeten kunnen uitloggen
3. Users moeten hun wachtwoord kunnen reseten
4. Users moeten hun account aan kunnen passen
5. Users moeten hun account kunnen verwijderen
6. Users moeten kunnen registreren

Epic 2:

1. Users moeten een applicatie aan kunnen maken
2. Users moeten een applicatie kunnen verwijderen
3. Users moeten een applicatie kunnen updaten
4. Users moeten hun eigen applicaties kunnen zien'

Epic 3:

1. Users moeten kunnen inloggen
2. Users moeten kunnen uitloggen
3. Users moeten hun wachtwoord kunnen reseten
4. Users moeten hun account aan kunnen passen
5. Users moeten hun account kunnen verwijderen
6. Users moeten kunnen registreren
7. Users moeten een applicatie aan kunnen maken
8. Users moeten een applicatie kunnen verwijderen
9. Users moeten een applicatie kunnen updaten
10. Users moeten hun eigen applicaties kunnen zien
11. Users moeten de logs kunnen zien van hun applicatie
12. Users moeten de env variabelen aan kunnen passen
13. Users moeten de metrics van hun applicaties kunnen zien
14. Users moeten hun applicaties kunnen managen

Epic 4:

1. Users moeten de logs kunnen zien van hun applicatie
2. Users moeten de env variabelen aan kunnen passen
3. Users moeten de metrics van hun applicaties kunnen zien
4. Users moeten hun applicaties kunnen managen

Non functional requirements

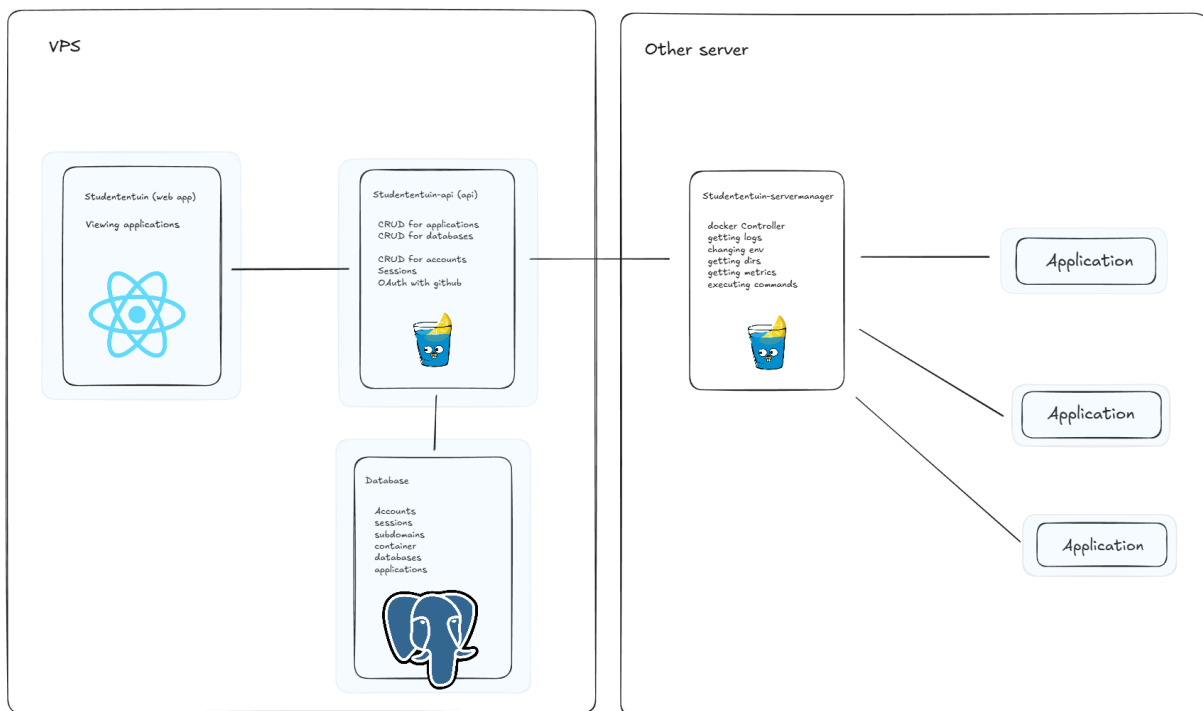
1. Het veilig maken van alle endpoints en het veilig maken van de frontend
2. Backends hebben een schaalbare en onderhoudbare architectuur
3. Frontend gebruikt een lage architectuur
4. De apis zijn geschreven in GO
5. De frontend is gemaakt in react met typescript
6. De database gebruikt postgres
7. We maken gebruik van een ORM
8. We proberen packages minimaal te houden en pinnen deze

Designing

UI

Bruh dat doet ai ik ben lui

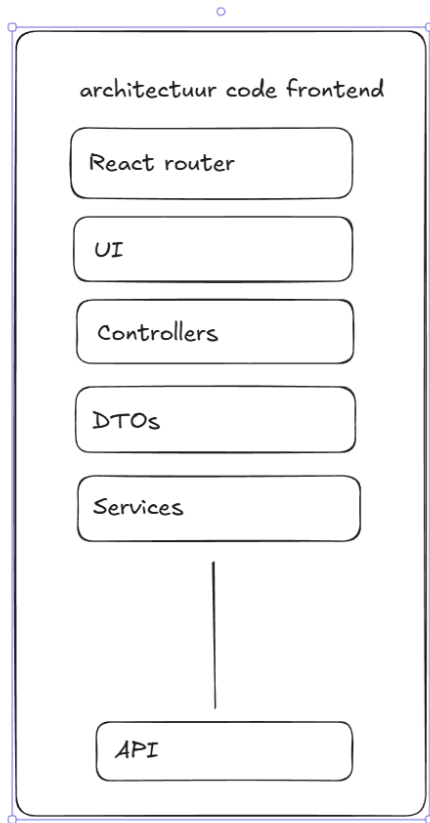
Architectuur



Dit is de architectuur van de applicatie. De verschillende grote rechthoeken zijn verschillende servers mijn main site zal (voor nu) gehost worden op een vps warin alles gecontainerized is (te zien aan de blauwe gloed eromheen). Onze server manager word op een aparte server gehost en deze heeft full access zodat hij ander applicaties aan en uit kan zetten dit gaan we doen door de applicaties met een base te containerizen en hierin de applicatie code te pullen en ze te builden en starten in de docker containers.

Je kan hier ook zien wat we gebruiken overal voor. Voor de fronend gebruiken we react voor de backend apis go met de gin framework en de db gebruikt postgres.

Architectuur frontend



Zoals je in het plaatje hierboven kan zien is het plan om een simpele lage structuur in de frontend te gebruiken waar boven aan je de react router hebt die bepaalt welke pagina wordt ingeladen daarna heb je de UI waar de pagina geladen wordt in de UI code wordt een connectie gemaakt met de controllers die de data ophaalt en mee stuurt voordat data in de services komt moet het eerst door de DTOs voor een extra laag data validatie. Dan komt het in de services en die zal een request sturen naar de API.